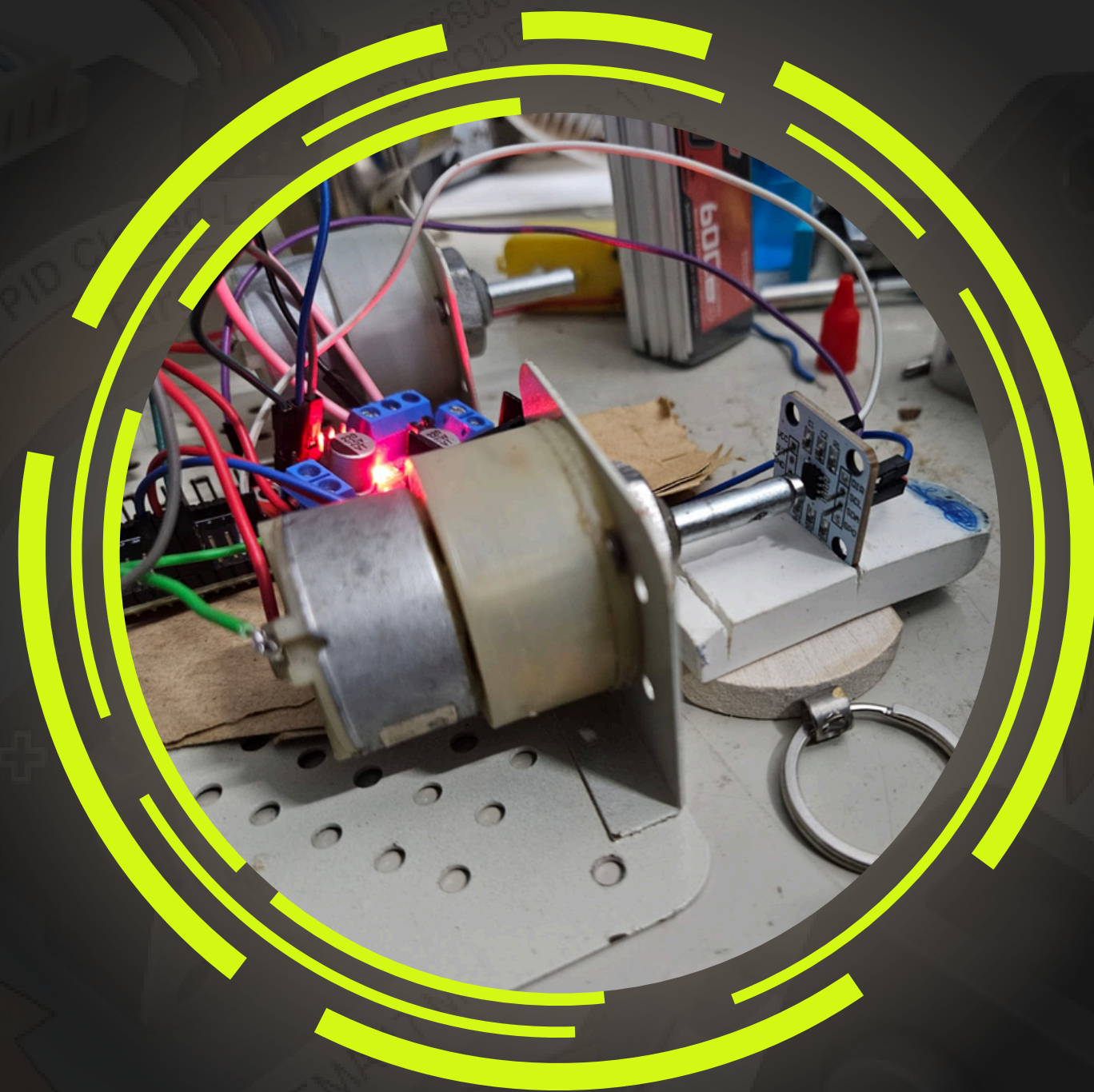


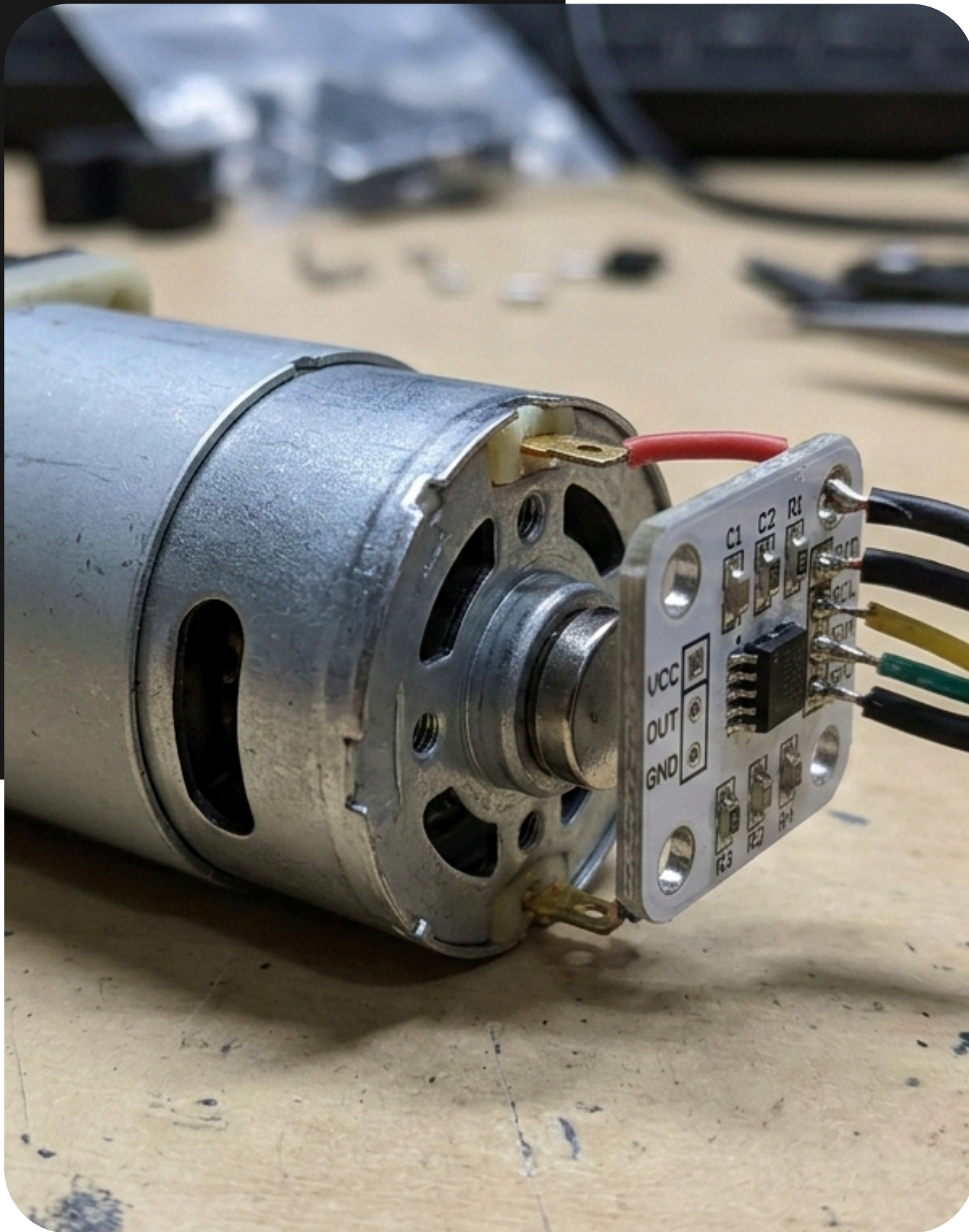
POSITION CONTROL OF ROBOTIC JOINT



BYOCS

TEAM MEMBERS

LALITH B	- CB.EN.U4ARE24021
SANJAY S	- CB.EN.U4ARE24045
SAI SHARBESH	- CB.EN.U4ARE24042
VISHAL KRISHNAA R	- CB.EN.U4ARE24057
SABARI POORNESH S	- CB.EN.U4ARE24040



INTRODUCTION

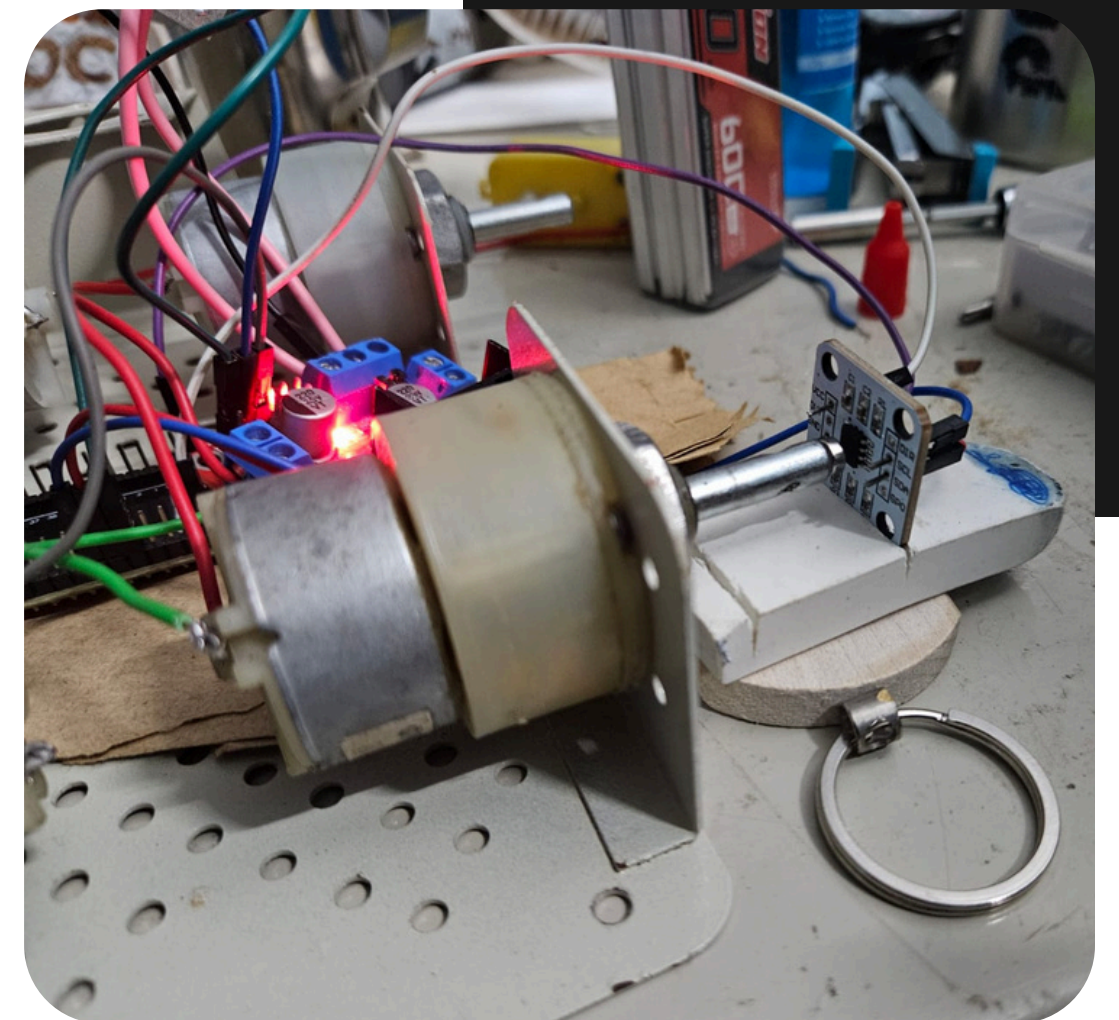
- This project implements a PID-based closed-loop system for precise DC motor position control.
- By integrating an AS5600 12-bit magnetic encoder with an ESP32-S3, it transforms a standard continuous-rotation motor into an intelligent "smart joint".
- The ESP32-S3 continuously monitors absolute angle feedback to calculate positional errors.

PROJECT OBJECTIVES

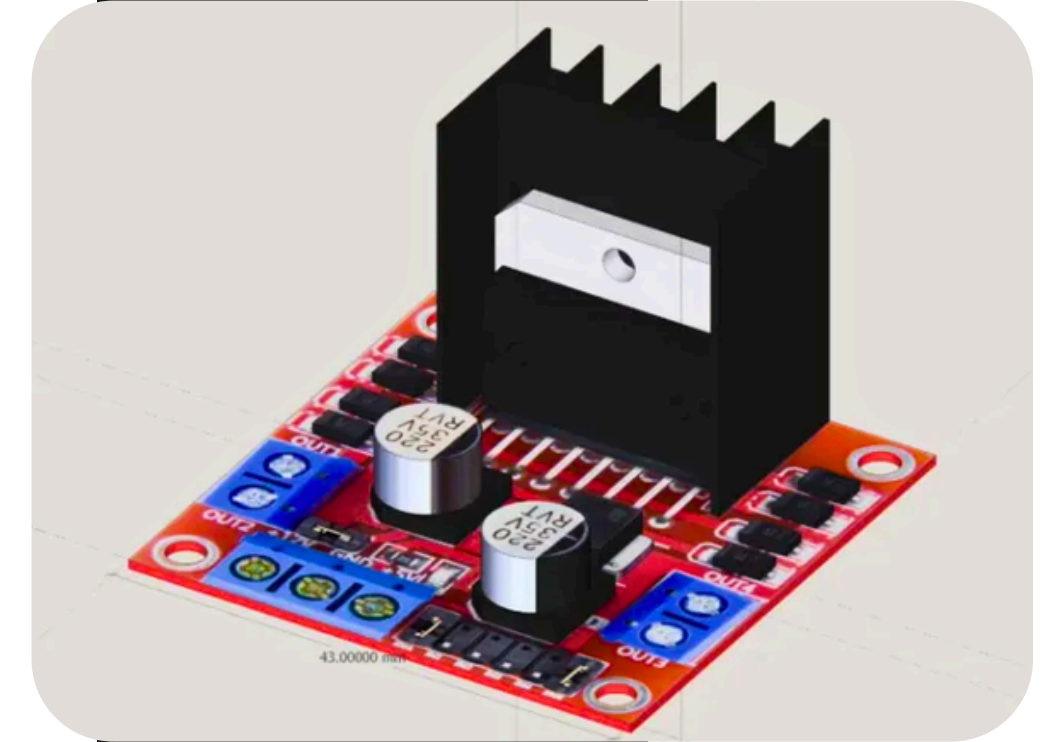
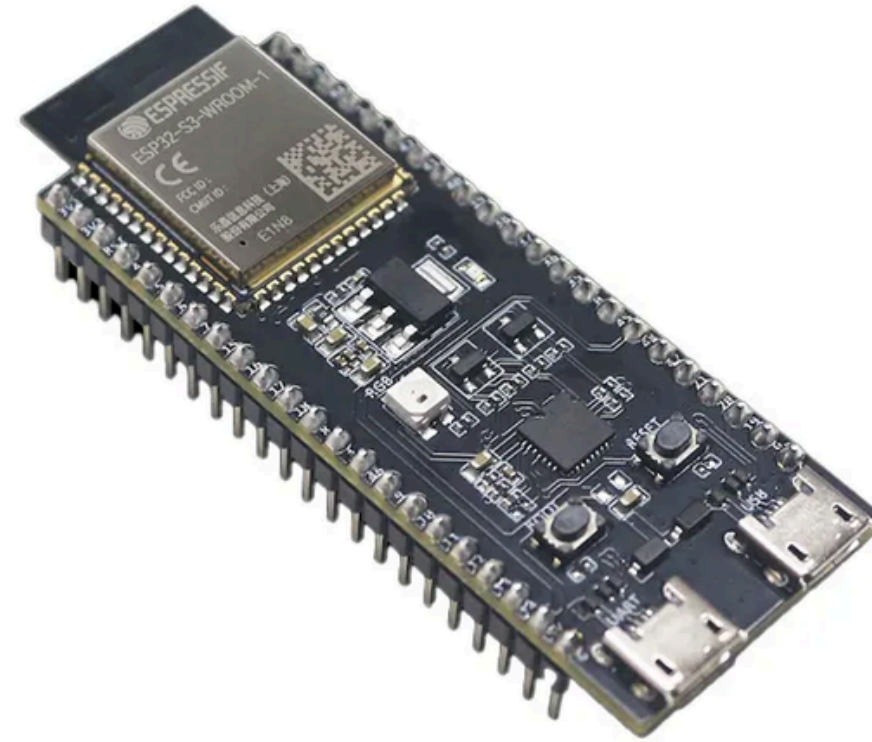
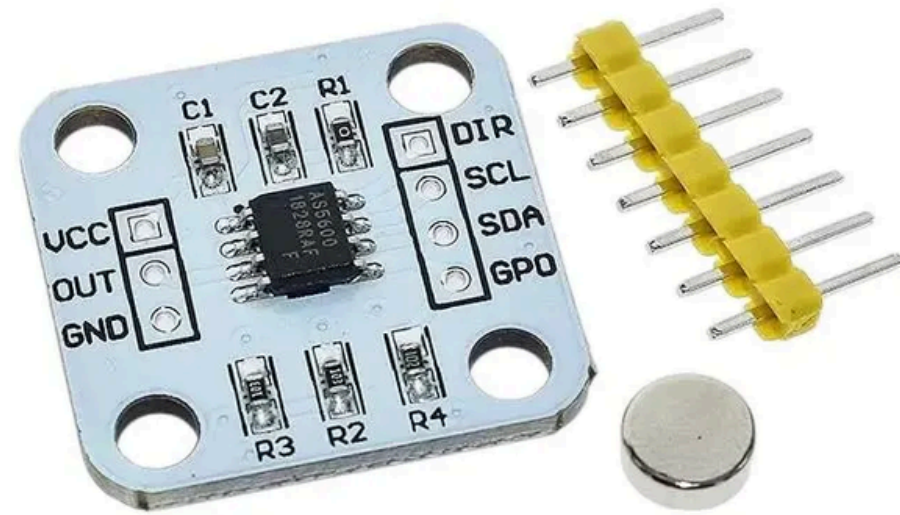
The Problem: Standard DC motors lack positional feedback, preventing an open-loop controller from confirming angle targets or correcting for inertial drift and gearbox "slop" (backlash).

The Objective: Create a high-precision robotic joint by adding a cost-effective, modular closed-loop controller that uses real-time magnetic feedback (negative control) to eliminate inertia drift and mechanical jitter.

Hardware Implementation: The system combines an ESP32-S3 microcontroller (the brain) for the PID loop, an AS5600 12-bit magnetic encoder (the eyes) for frictionless feedback, and an L298N driver (the muscle) for motor control.

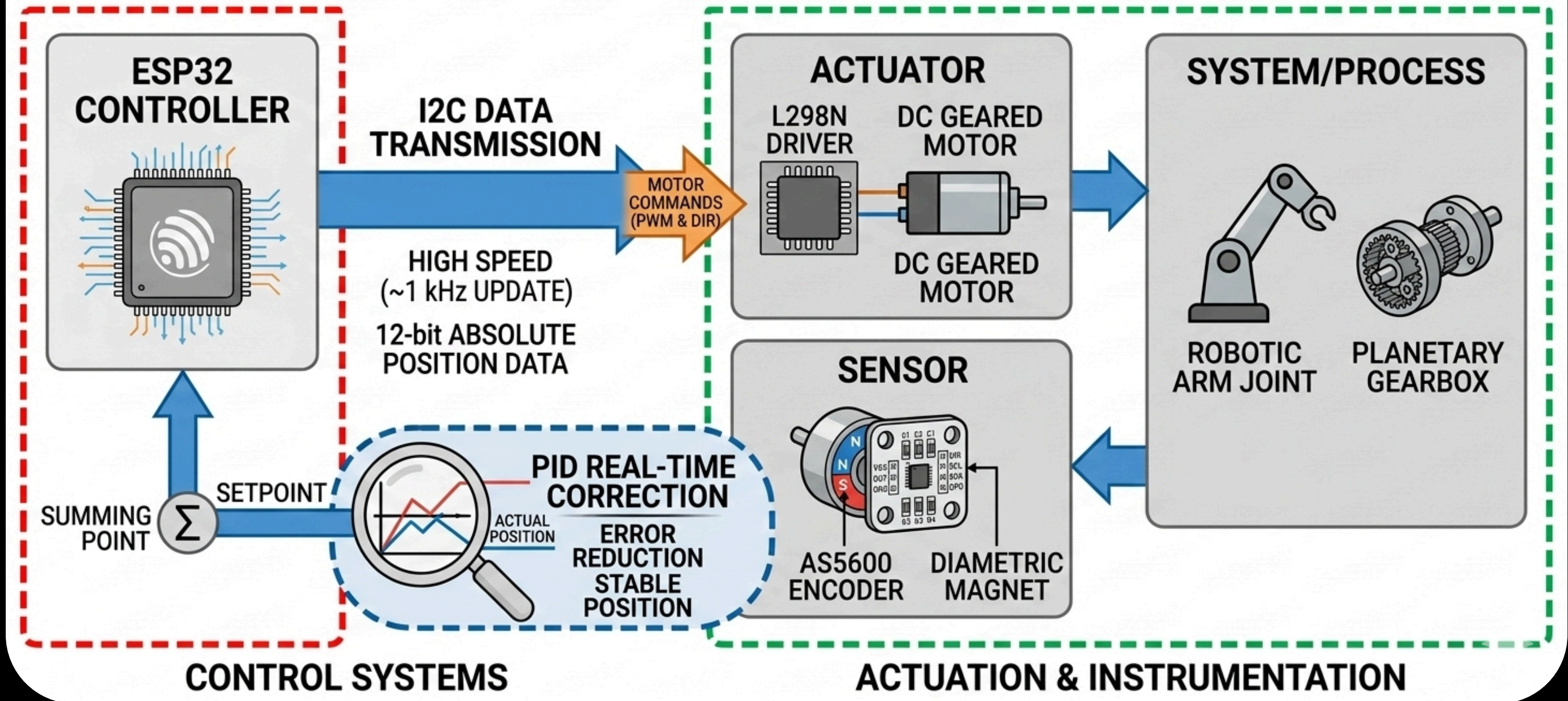


Process Fabrication



System Architecture

INTEGRATED CLOSED-LOOP PID ACTUATION SYSTEM (DC MOTOR)



Sensor Selection

CALIBRATION (AS5600)

Absolute Feedback

Unlike standard encoders, it remembers its exact angle even after a power cut.

I2C Interface

Digital communication is highly resistant to electrical noise from the motor.

Non-Contact (Magnetic)

No physical contact means zero friction and infinite life, unlike potentiometers that wear out.

High Precision

12-bit resolution provides 4,096 positions per turn (0.087° accuracy), enabling "Smooth Rotate" performance.



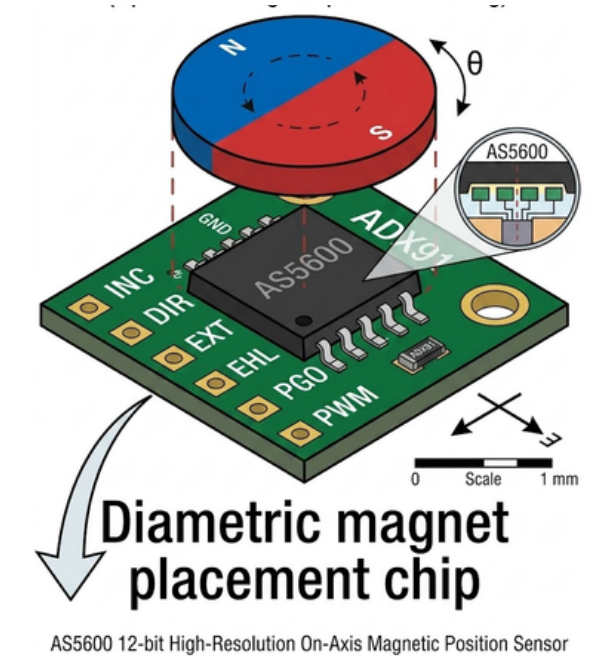
Principle of Operation

Hall Effect

The chip contains an array of Hall-effect sensors that detect the angle of a diametrically magnetized magnet.

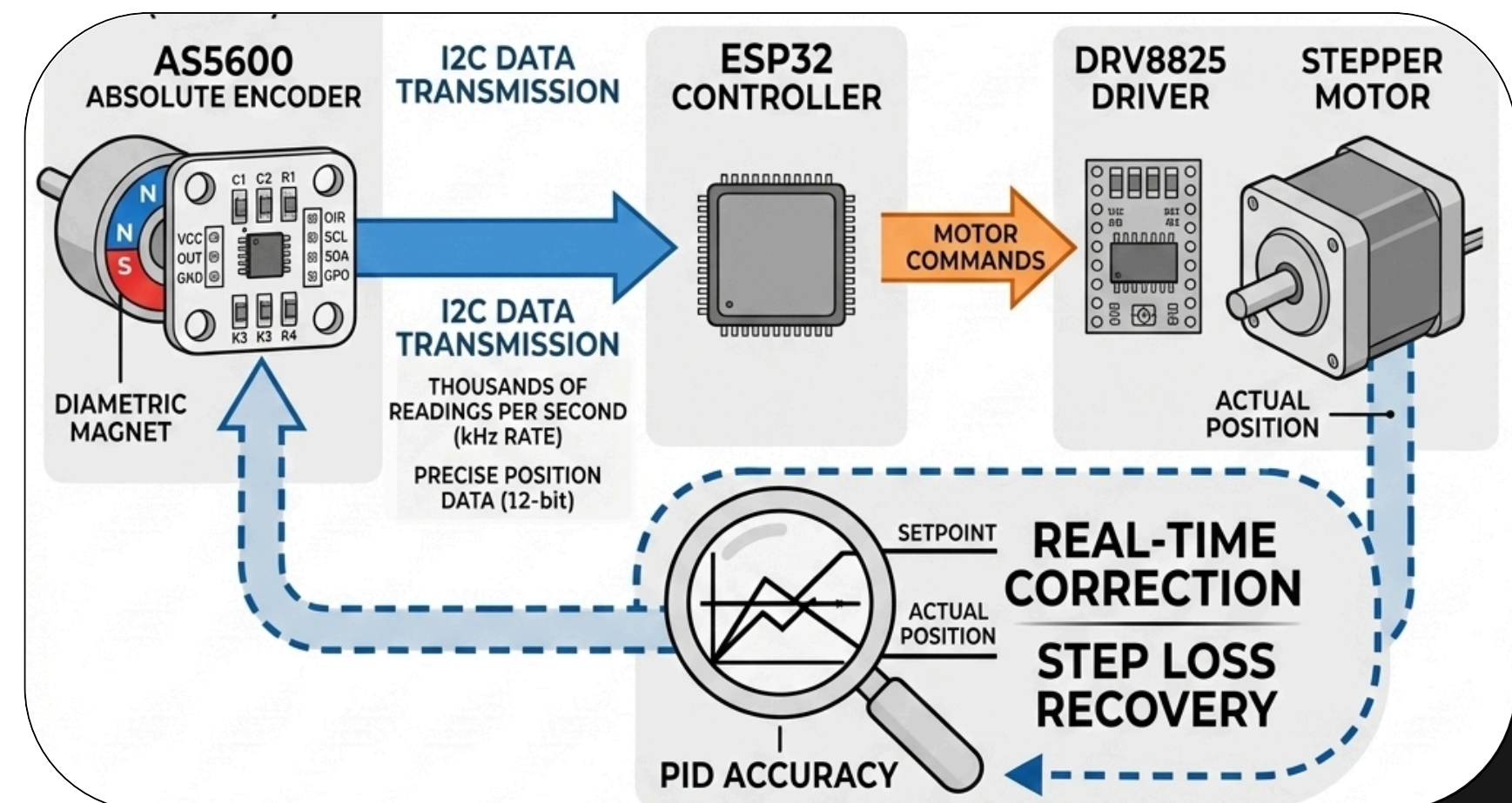
DSP Processing

An internal processor converts the rotating magnetic field lines into a digital angle in real-time.



* It sends data via I2C to the ESP32 thousands of times per second to keep the PID loop accurate.

$$\text{Angle in Degrees} = \frac{\text{Raw} \times 360^\circ}{4096}$$



Actuator Selection

CALIBRATION (DC MOTOR & L298M)

DC MOTOR

A gearbox is attached to the motor to increase the torque and improve the control resolution.

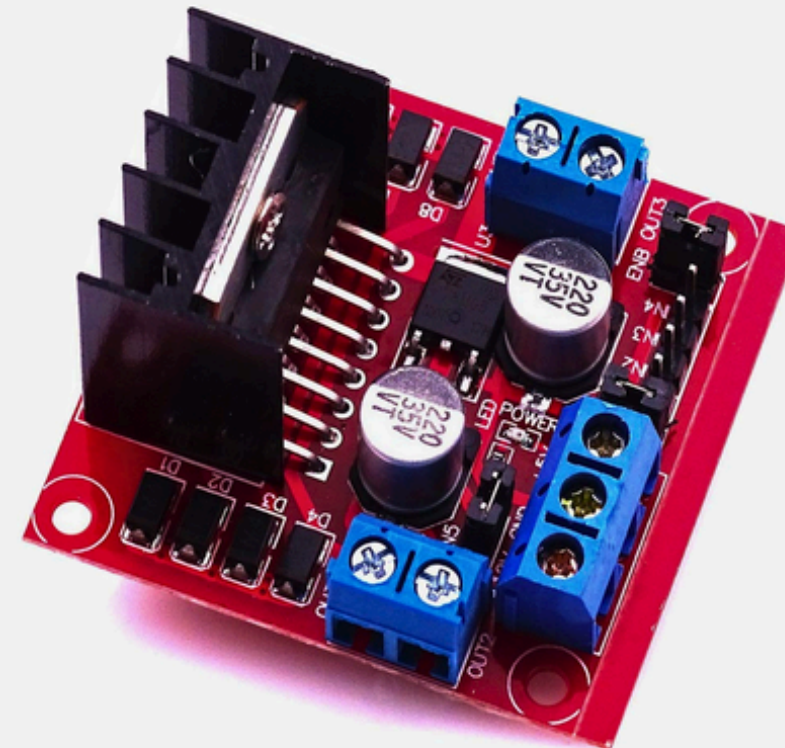
This helps the motor handle loads and maintain stable motion in the robotic joint.

The motor converts electrical energy into mechanical rotation, which moves the robotic joint to the desired position.”

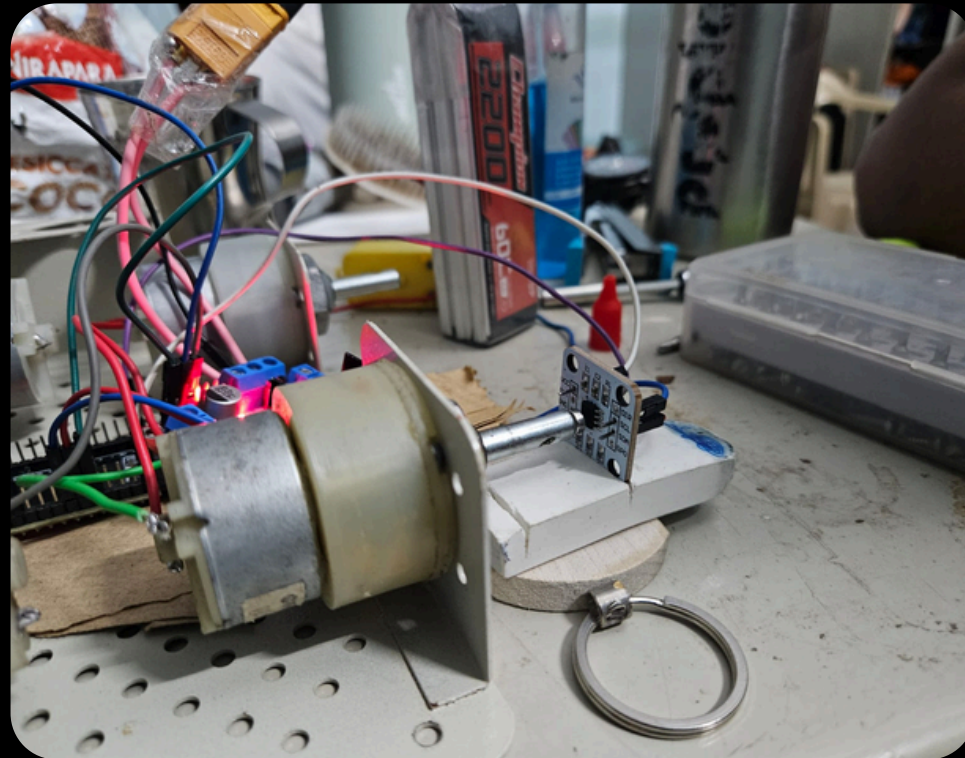
L298N

The ESP32 microcontroller produces low-voltage logic signals, but the motor requires higher current to operate.

The L298N driver acts as an interface, converting these control signals into sufficient voltage and current to drive the motor. It also allows us to control the direction and speed of the motor using PWM signals, which is important for implementing PID-based position control.”



DC MOTOR



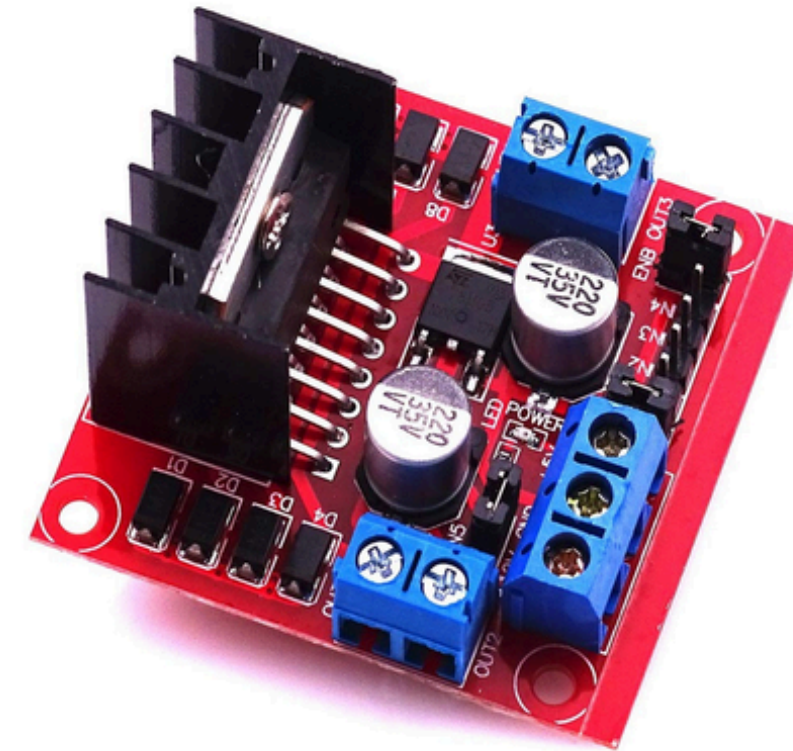
Advantages

Smooth Continuous Rotation:

- DC motors provide smooth rotation, which helps achieve stable movement when controlled with PID.

Compatible with Encoder Feedback:

- The motor works well with the AS5600 encoder, enabling accurate position measurement for closed-loop control.

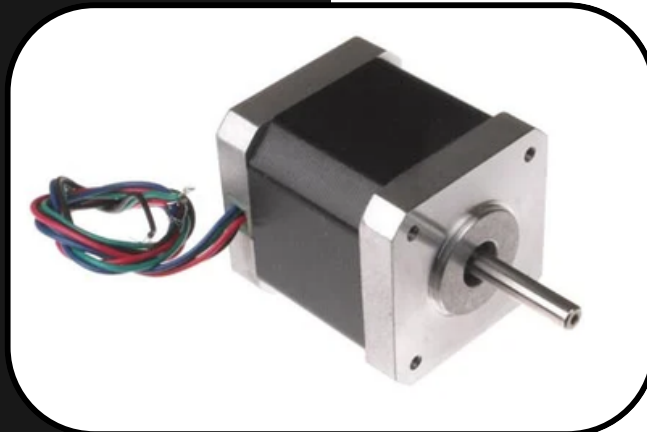


L298N

Advantages

The **L298N** acts as an **H-bridge driver**, allowing the controller to regulate the voltage and current supplied to the motor.

It enables us to control both the **direction and speed** of the DC motor using PWM signals, which is necessary for implementing PID-based position control.



WHY NOT STEPPER MOTOR ?

Open Loop Operation

Stepper motors typically operate in open-loop control, meaning the controller sends steps but cannot confirm if the motor actually reached the desired position. This can cause position errors if steps are missed.

Limited Speed Performance

Stepper motors perform well at low speeds but lose torque at higher speeds, which can limit the responsiveness of the robotic joint.



Higher Power Consumption

Stepper motors consumes current continuously, even when holding a position, which reduces overall system efficiency.

Controller Implementation

(ESP32-S3)

The **ESP32-S3** acts as the "brain." In a closed-loop system, the controller must simultaneously read the sensor, calculate **PID** (*Proportional-Integral-Derivative*) logic, and generate high-frequency pulses for the motor.

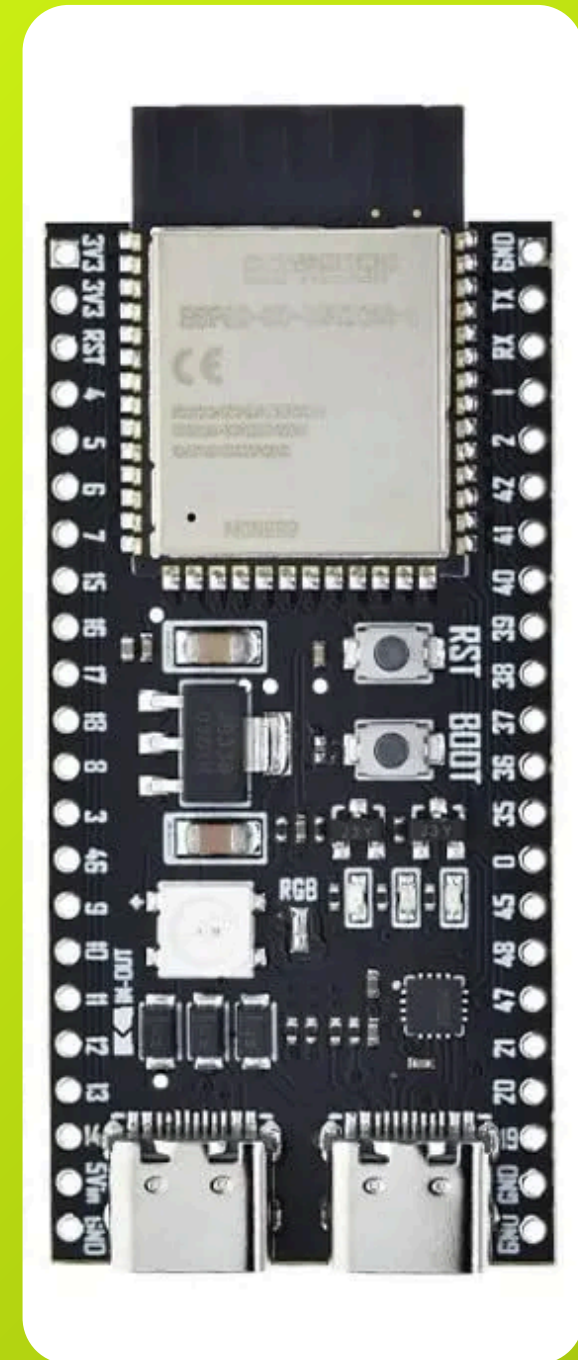
Properties

Dual-core 240MHz processor, multiple hardware Timers, and dual I2C/UART buses.

Advantages

Speed: Much faster than Arduino Nano/Uno, enabling frequent PID updates for smooth rotation.

Dual I2C: Communicates with the encoder (AS5600) and master controller on separate, dedicated buses—eliminating data collisions.



PID Algorithm & Logic

PID Discrete Implementation:

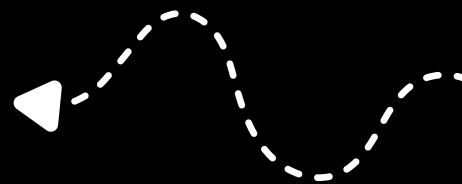
$$Output = (K_p \cdot \text{error}) + (K_i \cdot \text{integral}) + (K_d \cdot \text{derivative})$$

Proportional	Kp	The Muscle	Increases speed and reduces rise time; too high causes overshoot
Integral	Ki	The Precision	Eliminates steady-state error; too high causes instability (shaking/oscillation)
Derivative	Kd	The Brake	Predicts error and provides damping; reduces overshoot and oscillation

PID VALUES OF STABLE SYSTEM

Kp=1.2 KI=0 KD=0.35

```
integral += error * dt;  
integral = constrain(integral, -100, 100);  
derivative = (error - prevError) / dt;  
float output = Kp*error + Ki*integral + Kd*derivative;  
prevError = error;
```

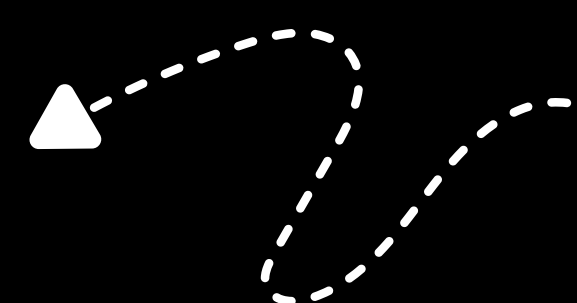
 **Anti-windup**

Actuator Calibration & Deadband



```
pwm = (int)abs(output);  
if (abs(error) < SLOW_ZONE) pwm = pwm * 0.55;  
pwm = constrain(pwm, 0, MAX_PWM);  
if (pwm > 0 && pwm < DEADBAND) pwm = DEADBAND;
```

```
float shortestError(float target, float current) {  
    float e = target - current;  
    if (e > 180) e -= 360;  
    if (e < -180) e += 360;  
    return e;  
}
```

 **Shortest Path
Logic (0-360°
Wrap-around)**

ENCODER 360°



135.0° ACTUAL + 180.0°

MOTION CONTROL

CUSTOM ANGLE (deg)

180.0 GO ▶

INCREMENT / DECREMENT

-45 -30 -1 +1 +30 +45

QUICK PRESETS

-90° -45° 0° 45° 90°

⦿ ZERO HERE

🛑 STOP

CURRENT TARGET: 180.0°

PID TUNING

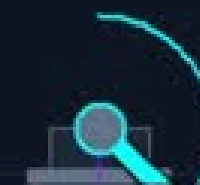
Kp PROPORTIONAL 1.2 SET

Ki INTEGRAL 0.0 SET

Kd DERIVATIVE 0.35 SET

SEND ALL PID

ARM VISUALIZATION



GRAPH 1 - SYSTEM RESPONSE (Target / Actual / Error)



GRAPH 2 - PID INTERNALS (P / I / D / PWM)



SERIAL LOG

✓ Reached 132.8°
→ a180.0
→ Target: 180.00°

System Integration

PID Controller Implementation and Tuning :

PID Control Equation

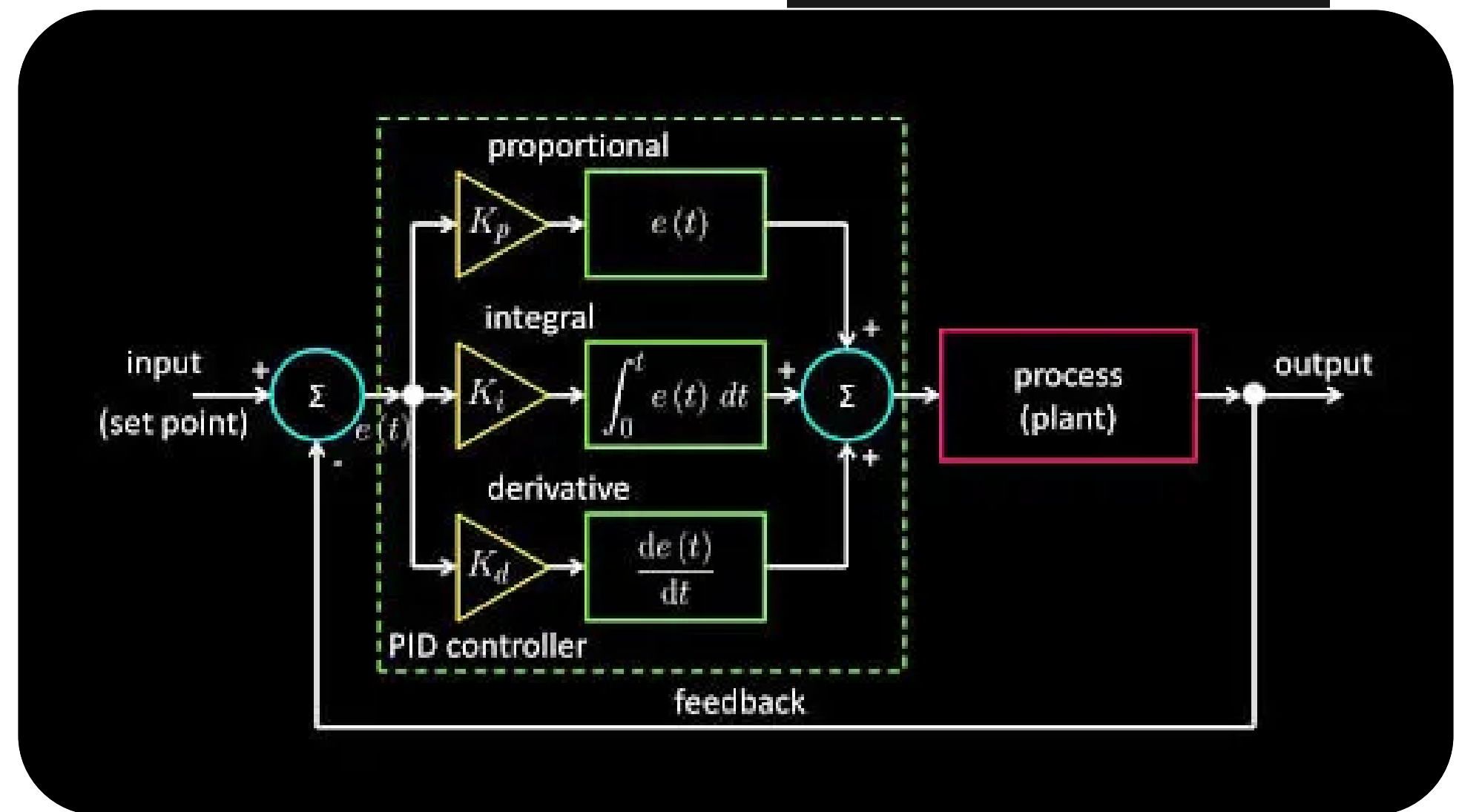
$$u(t) = K_p \left(e(t) + \frac{1}{T_i} \int e(t) dt + T_d \frac{de(t)}{dt} \right)$$

Tuning Method

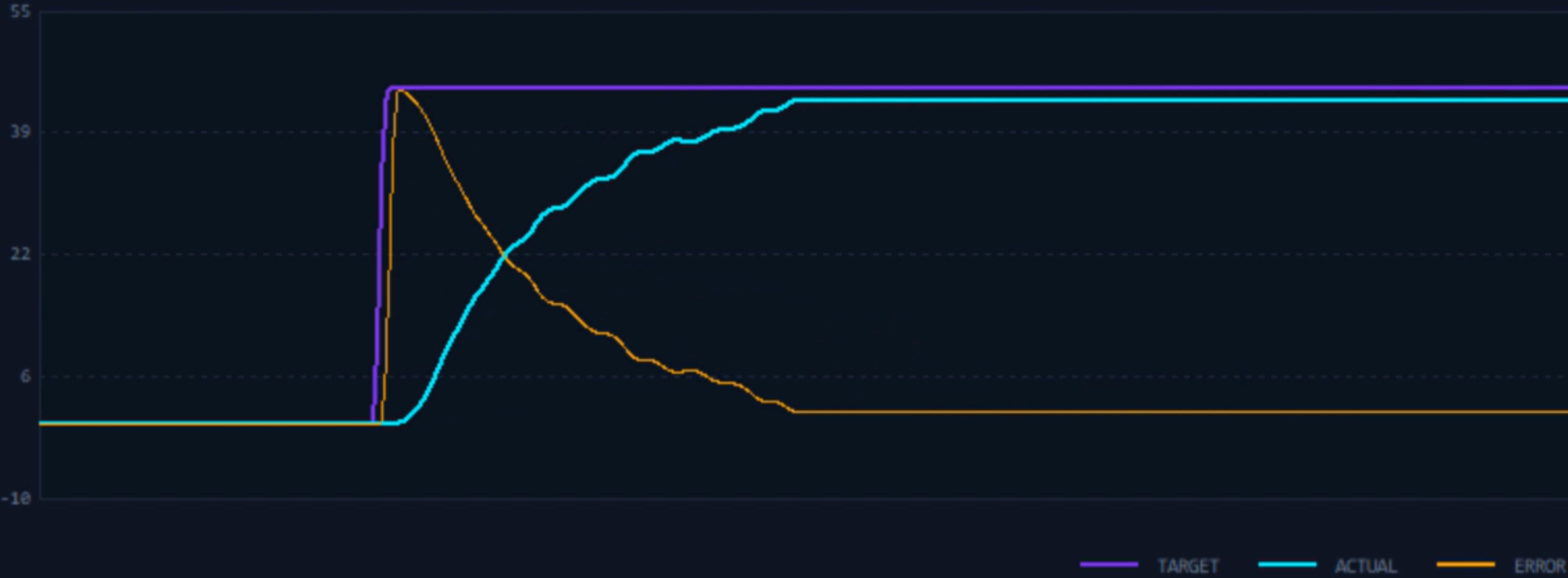
- Parameters tuned experimentally
- Kp increased to improve response speed
- Kd added to reduce overshoot
- Ki kept zero to avoid integral windup

Additional Improvements

- Deadband compensation
- Slow zone control near target
- PWM limiting for smooth stopping



GRAPH 1 – SYSTEM RESPONSE (Target / Actual / Error)



GRAPH 2 – PID INTERNALS (P / I / D / PWM)



SYSTEM RESPONSE

Parameter Value

K_p -1.2

K_i - 0

K_d -0.35

STEP INPUT TEST

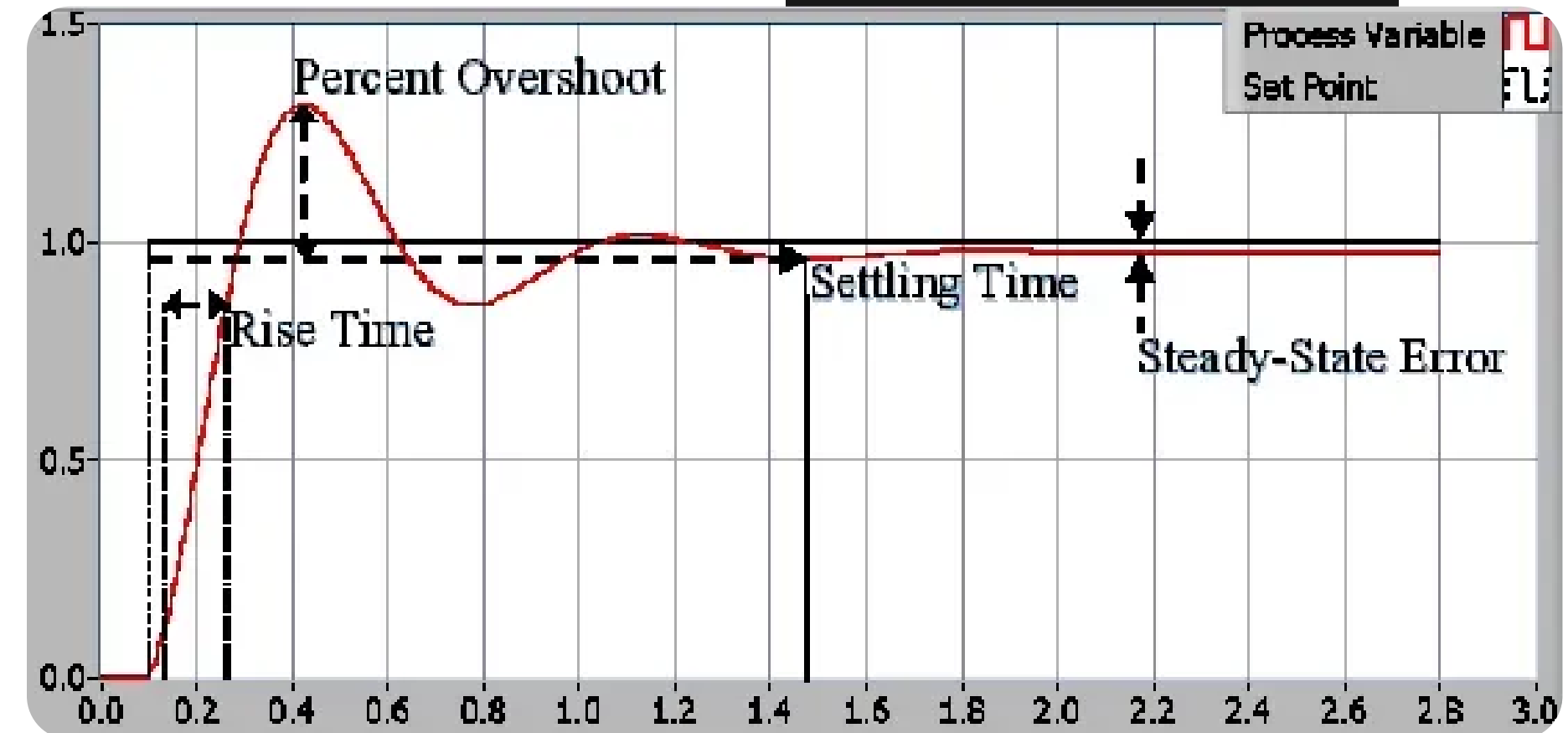
Target position changed from 0° to 40°

RESPONSE CHARACTERISTICS

- Actual angle follows the target position
- Small overshoot occurs due to motor inertia
- Error decreases gradually over time
- System reaches steady state smoothly

Observation

The PID controller successfully tracks the desired position with minimal oscillations



Performance Evaluation

Rise Time

Rise time is the time required for the output to go from 10% to 90% of the target value. The system reaches this range in approximately 0.8 seconds, for example target 40 deg

Overshoot

Overshoot occurs when the output goes beyond the target value before settling. Overshoot is approximately: 5% This small overshoot is acceptable

Settling Time

Settling time is the time required for the output to stay within a small tolerance band around the target.

In this system the tolerance band is: $\pm 2.5^\circ$

The system settles within approximately: 1.8 seconds

This indicates the system stabilizes quickly.



Steady State Error

Steady-state error is the final difference between the target and actual position after the system has settled.

Because of the high resolution encoder feedback, the steady-state error is: less than 1 degree

This shows the system achieves very accurate positioning.

Stability

A control system is considered stable if the output eventually settles to the target value and does not oscillate continuously.

From the graph we observe that:

- the output converges to the target
- oscillations decay over time
- the error approaches zero

Therefore the system is stable

**Thank
You!**